

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«Казанский национальный исследовательский технический университет им.
А.Н. Туполева – КАИ»

Институт компьютерных технологий и защиты информации
Кафедра Систем Информационной Безопасности

Лабораторная работа № 2

Дисциплина: «Аппаратные средства телекоммуникационных систем»

Работу выполнил

Студент гр. 4404

Романов Е.М.

Проверил

Шигабутдинов М.И.

Казань, 2026

Задание 4

Напишите программу перемножения больших матриц. Сравните время выполнения последовательной и параллельной программы на 2, 4, 8 потоках (процессорах).

Программа принимает из командной строки два аргумента: размер матрицы N и число потоков. Выделяются четыре матрицы $N \times N$: A , B - входные; C_{seq} , C_{par} — результаты последовательного и параллельного умножения.

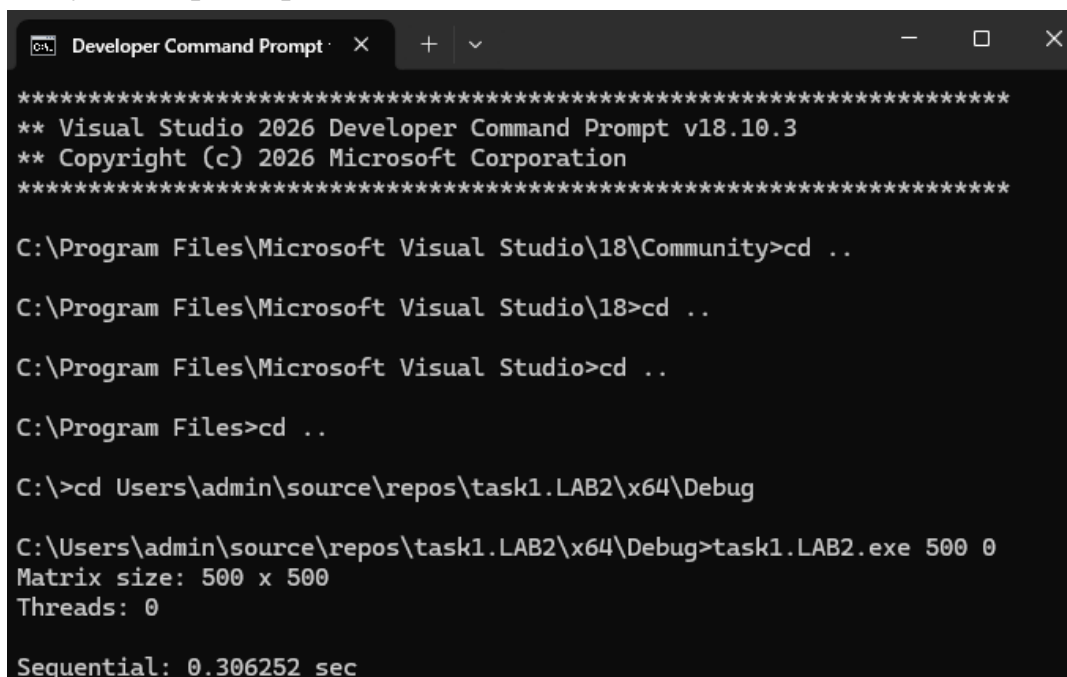
Матрицы A и B инициализируются по формуле $m[i][j] = (i + j) / N$.

Последовательное умножение выполняется классическим тройным циклом:

```
for (i = 0; i < N; i++)
    for (j = 0; j < N; j++) {
        double sum = 0.0;
        for (k = 0; k < N; k++)
            sum += A[i][k] * B[k][j];
        C[i][j] = sum;
    }
```

Параллельное умножение отличается только директивой `#pragma omp parallel for private(j, k) shared(A, B, C, N)`. Распараллелен внешний цикл по i — каждая итерация обрабатывает свою строку результата $C[i][*]$, поэтому зависимости по данным нет.

Запуск 1 — размер 500x500, число потоков 0.



```
Developer Command Prompt v18.10.3
** Visual Studio 2026 Developer Command Prompt v18.10.3
** Copyright (c) 2026 Microsoft Corporation
*****

C:\Program Files\Microsoft Visual Studio\18\Community>cd ..
C:\Program Files\Microsoft Visual Studio\18>cd ..
C:\Program Files\Microsoft Visual Studio>cd ..
C:\Program Files>cd ..
C:\>cd Users\admin\source\repos\task1.LAB2\x64\Debug
C:\Users\admin\source\repos\task1.LAB2\x64\Debug>task1.LAB2.exe 500 0
Matrix size: 500 x 500
Threads: 0
Sequential: 0.306252 sec
```

Запуск 2 – размер 500x500, число потоков 2.

```
C:\Users\admin\source\repos\task1.LAB2\x64\Debug>task1.LAB2.exe 500 0
Matrix size: 500 x 500
Threads: 0

Sequential: 0.306252 sec

C:\Users\admin\source\repos\task1.LAB2\x64\Debug>task1.LAB2.exe 500 2
Matrix size: 500 x 500
Threads: 2

Sequential: 0.313650 sec
Parallel (2 threads): 0.159523 sec
Speedup: 1.966
Result check: OK
```

Запуск 3 – размер 500x500, число потоков 4.

```
C:\Users\admin\source\repos\task1.LAB2\x64\Debug>task1.LAB2.exe 500 2
Matrix size: 500 x 500
Threads: 2

Sequential: 0.313650 sec
Parallel (2 threads): 0.159523 sec
Speedup: 1.966
Result check: OK

C:\Users\admin\source\repos\task1.LAB2\x64\Debug>task1.LAB2.exe 500 4
Matrix size: 500 x 500
Threads: 4

Sequential: 0.304954 sec
Parallel (4 threads): 0.097762 sec
Speedup: 3.119
Result check: OK
```

Запуск 4 – размер 500x500, число потоков 8.

```
C:\Users\admin\source\repos\task1.LAB2\x64\Debug>task1.LAB2.exe 500 4
Matrix size: 500 x 500
Threads: 4

Sequential: 0.304954 sec
Parallel (4 threads): 0.097762 sec
Speedup: 3.119
Result check: OK

C:\Users\admin\source\repos\task1.LAB2\x64\Debug>task1.LAB2.exe 500 8
Matrix size: 500 x 500
Threads: 8

Sequential: 0.313086 sec
Parallel (8 threads): 0.076037 sec
Speedup: 4.118
Result check: OK
```

Потоки	Последовательное время	Параллельное время	Ускорение
0	0,306252	-	-
2	0,313650	0,159523	1,966
4	0,304954	0,097762	3,119
8	0,313086 с	0,076037 с	4,118

Вывод: по результатам замеров видно, что параллельная версия превосходит последовательную по времени выполнения во всех конфигурациях. Ускорение демонстрирует устойчивый рост с увеличением числа потоков, однако этот рост не является линейным. На двух потоках ускорение практически достигает двукратного значения, тогда как дальнейшее увеличение числа потоков до 4 и 8 даёт всё меньший прирост эффективности.

Наблюдаемое отклонение от идеального ускорения объясняется действием трёх основных факторов. Во-первых, это наличие последовательной части в программе — инициализация матриц, вывод результатов, а также операции создания и завершения потоков. Во-вторых, с ростом числа потоков увеличиваются расходы на создание и синхронизацию потоков. В-третьих, подсистема памяти становится узким местом: одновременные обращения множества потоков к общим массивам А и В превышают пропускную способность шины памяти, что приводит к снижению эффективности распараллеливания.